

PROJECT REPORT

Pothole Detection

Image Classification Project for Detecting Potholes and Normal Roads

1. Project Description and Details

This project is a simple image classification system that detects whether a road image contains a pothole or shows a normal road. The implementation uses MobileNetV2 with transfer learning. MobileNetV2 is loaded with ImageNet weights, while a small custom classification head is added for the two project classes.

Project objective: Classify a road image into one of two categories: Normal Road or Pothole.

Input: A color road image resized to 224 x 224 pixels before classification.

Output: Predicted class and confidence percentage.

Model approach: A frozen MobileNetV2 feature extractor is followed by Global Average Pooling, Dropout, and a two-class Softmax output layer.

2. Project Workflow

- **Dataset preparation:** Road images are organized into Plain (normal road) and Pothole classes.
- **Dataset split:** Predefined train/validation/test folders are supported. If only class folders are available, the program creates an 80% training, 10% validation, and 10% test split.
- **Image preprocessing:** Images are decoded as three-channel images and resized to 224 x 224 pixels.
- **Data augmentation:** Random horizontal flipping, rotation, and zoom are applied during training.
- **Transfer learning:** MobileNetV2 with ImageNet weights is used as a frozen feature extractor.
- **Classification head:** GlobalAveragePooling2D, Dropout (0.3), and a Softmax Dense layer classify the image.
- **Training:** Adam optimizer and categorical crossentropy are used for 20 epochs.
- **Evaluation:** Accuracy, precision, recall, F1 score, classification report, and confusion matrix are generated.
- **Prediction:** predict.py loads the saved model and predicts the class of a user-provided image.

3. Software Requirements

Requirement	Details
Operating system	Windows / Linux / macOS with a Python and TensorFlow-compatible environment.
Programming language	Python 3.x
Deep learning framework	TensorFlow / Keras
Numerical computing	NumPy
Visualization	Matplotlib and Seaborn
Evaluation	Scikit-learn
Internet	May be required during first training if MobileNetV2 needs to download ImageNet weights.
Hardware	CPU is sufficient for basic execution; a compatible GPU can accelerate training.

4. Dataset Requirements and Structure

The project expects a dataset folder containing two class folders named **Plain** and **Pothole**. The code also supports pre-existing train/valid/test arrangements.

Recommended structure:

```
dataset/  
- train/  
  - Pothole/  
  - Plain/  
- valid/  
  - Pothole/  
  - Plain/  
- test/  
  - Pothole/  
  - Plain/
```

If only Pothole and Plain folders are available, train.py splits each class into 80% training, 10% validation, and 10% testing. If train and test folders exist without valid, 10% of the training-folder images are used for validation.

5. Model and Training Configuration

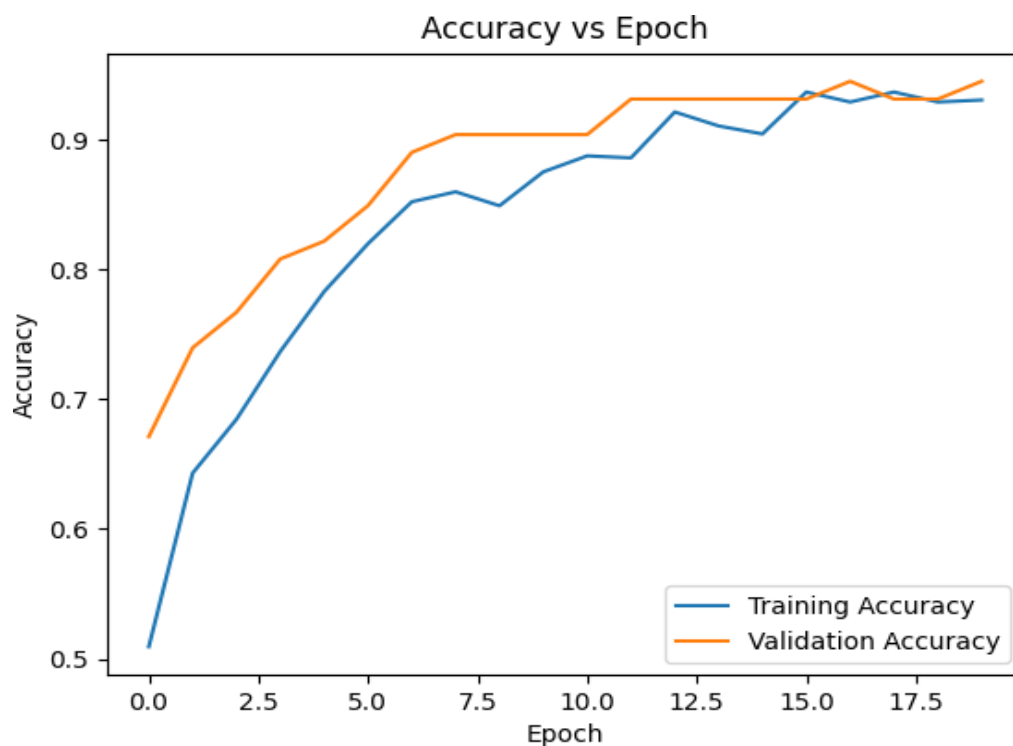
Parameter	Configured value
Image size	224 x 224
Batch size	32
Epochs	20
Learning rate	0.0001
Optimizer	Adam
Loss	Categorical Crossentropy
Base model	MobileNetV2, ImageNet weights, include_top=False
Base model training	Frozen
Dropout	0.3
Output activation	Softmax
Classes	Normal Road, Pothole
Random seed	42

6. Training and Evaluation Outputs

After training, the project saves the trained model as **pothole_mobilenetv2_model.keras** and generates accuracy and loss graphs. It also calculates accuracy, precision, recall, F1 score, a classification report, and a confusion matrix.

6.1 Accuracy vs Epoch

The supplied result screenshot shows training and validation accuracy increasing across the 20 epochs, with both curves reaching approximately the 0.90+ range toward the end of training.



7. Prediction / Deployment Procedure

The prediction script loads the saved Keras model, asks the user to enter an image path, checks that the path exists and points to a file, reads and decodes the image, resizes it to 224 x 224, and sends it to the model. The class with the highest

7.1 Prediction Results

The following examples demonstrate the prediction procedure using one normal-road image and one pothole image. Each input image is followed by the corresponding model output.

Normal Road Input Image



Model Output – Normal Road

```
Enter image path: "C:\Users\ASUS\OneDrive\Desktop\pothholedetection\OGG.jpg"  
Prediction : Normal Road  
Confidence : 99.62%
```

Pothole Input Image



Model Output – Pothole

```
Enter image path: "C:\Users\ASUS\OneDrive\Desktop\pothholedetection\gg.png"  
Prediction : Pothole  
Confidence : 99.41%
```

predicted probability is returned together with its confidence percentage.

Run the following command from the project root:

```
python predict.py
```

The script uses the class order **Normal Road, Pothole** and the saved model file name **pothole_mobilenetv2_model.keras**. Example output:

```
Prediction : Pothole
Confidence : 97.56%
```

8. Complete Code Path / Location

For pen-drive submission, keep all project files inside one project folder. The folder name can be changed as required.

Project structure:

```
Pothole_Detection_MobileNetV2/
- train.py
- predict.py
- pothole_mobilenetv2_model.keras
- requirements.txt
- README.md
- .gitignore
- accuracy.png
- loss.png (generated after training)
- confusion_matrix.png (recommended saved output)
- dataset/
```

The **dataset/** folder should contain the required training/testing images in the supported class-folder structure.

9. All Required Source Code Files

File	Purpose
train.py	Complete training pipeline: dataset handling, preprocessing, augmentation, MobileNetV2 model construction, training, model saving, plots, and evaluation.
predict.py	Loads the saved model and predicts Normal Road or Pothole for a user-provided image.
requirements.txt	Lists the Python packages required to run the project.
README.md	Contains project setup, dataset structure, installation, training, prediction, and result information.
.gitignore	Excludes the dataset, Python cache files, and compiled Python files from version-control tracking.

10. Supporting Files

- **pothole_mobilenetv2_model.keras**: Saved trained Keras model used by predict.py.
- **accuracy.png**: Supplied training/validation accuracy screenshot.
- **loss.png**: Generated by train.py after training.
- **confusion_matrix.png**: Recommended to save as an image after training because the current code displays the confusion matrix but does not save it.
- **dataset/**: Required runtime data if the evaluator wants to retrain the model.

11. Installation and Execution

Install dependencies:

```
pip install -r requirements.txt
```

Train the model:

```
python train.py
```

Run prediction:

```
python predict.py
```

For training, ensure the dataset is placed in the dataset folder. For prediction, ensure pothole_mobilenetv2_model.keras is in the same project root as predict.py.

12. Conclusion

The Pothole Detection project successfully demonstrates an image classification system for identifying whether a given road image represents a Normal Road or a Pothole. The system uses MobileNetV2 with transfer learning, allowing the project to make use of pretrained visual features while adding a custom classification layer for the two required classes.

The complete project workflow includes dataset preparation, image preprocessing, data augmentation, model training, evaluation, and image prediction. The input road images are resized to 224×224 pixels before being provided to the model. During training, augmentation techniques such as horizontal flipping, rotation, and zoom are used to improve the model's ability to handle variations in road images. The model is trained using the Adam optimizer and categorical crossentropy loss.

The prediction module provides a simple method for testing new road images. The user can provide an image path, after which the trained model processes the image and predicts whether the road is Normal Road or Pothole, along with a confidence value generated for that particular prediction. This makes the system useful for demonstrating how deep learning can be applied to automated road-condition analysis.

Overall, the project provides a practical foundation for automated pothole detection using deep learning and computer vision. It can be further improved by increasing the size and diversity of the dataset, including images captured under different weather and lighting conditions, adding additional road-condition categories, and developing a graphical or web-based interface for easier usage. Future versions could also integrate the system with mobile or vehicle-mounted cameras to support real-time road monitoring. The overall performance of the system will depend on the quality, size, and variety of the dataset used for training.

